

Bipartite Graph Network with Pair-Conditioned Message Passing and Cross-Step Memory for Flexible Job Shop Scheduling

Kaiyuan Li¹, Xungen Li^{1,2*}

¹School of Electronics and Information, Hangzhou Dianzi University, Hangzhou, 310018, China

²Pujiang Microelectronics and Intelligent Manufacturing Research Institute, Hangzhou Dianzi University, Jinhua, 322200, China

*Corresponding author's e-mail: lixg@hdu.edu.cn

Keywords: FLEXIBLE JOB SHOP SCHEDULING, DEEP REINFORCEMENT LEARNING, BIPARTITE GRAPH NEURAL NETWORK, RECURRENT MEMORY

Abstract

Flexible job shop scheduling is a fundamental production-planning problem in intelligent manufacturing systems. Existing graph-based deep reinforcement learning (DRL) methods often encode full operation-level graphs and perform message passing over operations that are not directly involved in the current decision. To address this issue, we propose an end-to-end DRL framework with a bipartite graph network and cross-step memory. The scheduling state is formulated as a compact bipartite graph connecting one candidate operation per job to eligible machines, with feasible edges encoding assignment-specific information. The proposed network employs bidirectional attention between the two node sets, guided by pair features to capture their interactions. A gated recurrent unit maintains the temporal context required for sequential scheduling decisions. Benchmark experiments show that the proposed method is competitive with recent DRL baselines and performs consistently across benchmark sets with different instance distributions.

1 Introduction

Advanced manufacturing systems require flexible and efficient production scheduling [1]. The flexible job shop scheduling problem (FJSP) captures this requirement by allowing each precedence-constrained operation to be processed on multiple eligible machines with different processing times. Solving an FJSP instance requires jointly determining operation sequences and machine assignments to minimize the makespan. The coupling of precedence constraints, machine competition, and alternative machine assignments makes the problem NP-hard [2]. Traditional solution approaches include exact and metaheuristic methods, which often trade solution quality against computational cost and require a new search for each instance [3].

Deep reinforcement learning (DRL) has been increasingly applied to FJSP because scheduling policies can be learned offline and rapidly applied to new instances. Recent studies have explored different state representations for capturing FJSP scheduling information [4]. A scheduling state must describe the evolving job progress, machine status, and available operation-machine assignments. Its representation therefore affects both the information available to the policy and the computational cost of decision-making. Vector-based methods encode candidate actions using fixed-length or handcrafted features, providing compact inputs for policy learning [5-7]. Graph-based methods organize precedence constraints, machine eligibility, and resource competition as graph structures [8-11]. However, full operation-level graphs include many operations that are not available for selection at the current decision step, increasing the size of the state

representation and the cost of message passing. Elaborate message-passing schemes may further increase the computational cost of online scheduling.

To address this limitation, we propose an end-to-end DRL framework based on a compact operation-machine bipartite graph. The main contributions are as follows: (1) a compact bipartite state representation that retains the current assignment alternatives; (2) a bipartite graph neural network with pair-conditioned bidirectional attention; and (3) GRU-based cross-step memory for maintaining scheduling context across decisions.

2 Problem Description

An FJSP instance consists of a set of jobs $\mathcal{J} = \{J_1, \dots, J_n\}$ and a set of machines $\mathcal{M} = \{M_1, \dots, M_m\}$. Each job J_i contains an ordered sequence of n_i operations, $(O_{i1}, \dots, O_{in_i})$. Each operation O_{ij} can be processed on one machine selected from its set of eligible machines $\mathcal{M}_{ij} \subseteq \mathcal{M}$. Its processing time $p_{ijk} > 0$ depends on the selected machine M_k . Let S_{ij} and C_{ij} denote its start and completion times, respectively.

A feasible schedule assigns each operation to an eligible machine and determines its start time. Operations are processed without preemption. If O_{ij} is assigned to M_k , then $C_{ij} = S_{ij} + p_{ijk}$. The precedence constraints require $S_{ij} \geq C_{i,j-1}$ for $j = 2, \dots, n_i$. Any two operations assigned to the same machine cannot overlap. The makespan is $C_{max} = \max_{1 \leq i \leq n} C_{i,n_i}$, and the objective is to minimize C_{max} .

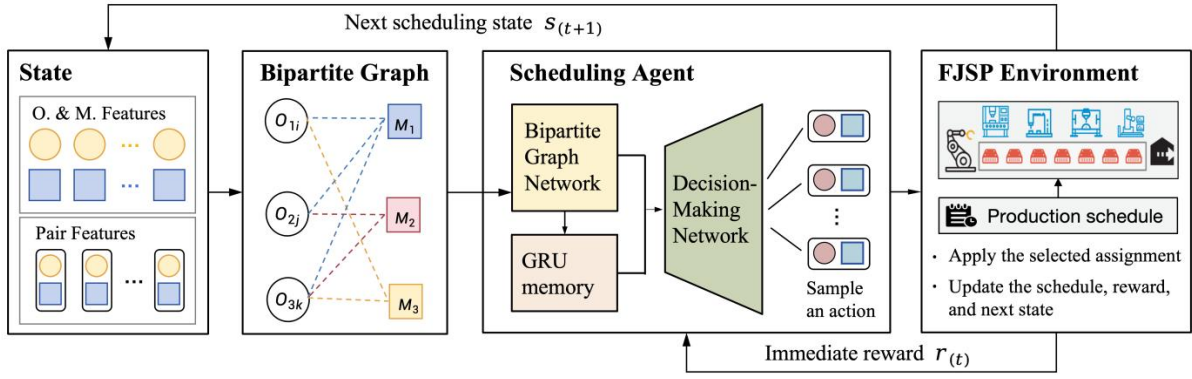


Figure 1 Overview of the proposed DRL framework for FJSP.

3 Methodology

We formulate FJSP scheduling as a sequential decision-making problem and solve it using a DRL framework trained with proximal policy optimization (PPO), as shown in Fig. 1.

3.1 Compact Bipartite Graph Representation

We formulate FJSP scheduling as a Markov decision process (MDP) that sequentially assigns operations to machines. As illustrated in Fig. 2, the compact bipartite graph is derived from the conventional disjunctive graph by retaining the first unscheduled operation of each unfinished job and its eligible machines. At decision step t , the graph is defined as $G_t = (\mathcal{V}_t^O, \mathcal{V}_t^M, \mathcal{E}_t)$, where $\mathcal{V}_t^O$ contains the retained candidate operations, $\mathcal{V}_t^M$ contains all machines, and $(o_i^t, M_k) \in \mathcal{E}_t$ if M_k is eligible to process o_i^t .

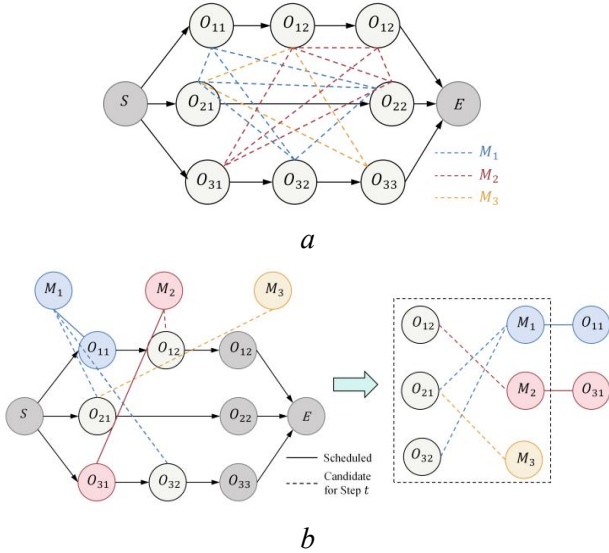


Figure 2. Derivation of the compact bipartite representation (a) Conventional disjunctive graph (b) Compact bipartite graph at decision step t

The graph is described by operation features $\mathbf{F}_t^O \in \mathbb{R}^{n \times 8}$, machine features $\mathbf{F}_t^M \in \mathbb{R}^{M \times 5}$, pair features $\mathbf{F}_t^P \in \mathbb{R}^{n \times M \times 6}$, and a dynamic mask for completed jobs and ineligible assignments. These features encode job and machine availability, remaining workload, processing times, and pairwise waiting times. By retaining at most n candidate

operations instead of all N operations, the operation-machine pair representation is reduced from $O(Nm)$ to $O(nm)$, while m is the number of machines.

Within this MDP, an action $a_t = (o_i^t, M_k) \in \mathcal{E}_t$ jointly selects a candidate operation and an eligible machine. The selected operation is appended to the corresponding machine sequence, after which the state is updated deterministically. The stepwise reward is defined as:

$$r_t = C_{LB}(s_t) - C_{LB}(s_{t+1}) \quad (1)$$

where C_{LB} is the maximum lower-bound completion time over all operations on the normalized time scale. With $\gamma = 1$, maximizing the cumulative reward in (1) minimizes the final makespan on the normalized time scale.

3.2 Bipartite Graph Neural Network

As shown in Fig. 3, operation, machine, and pair features are mapped to embeddings $\mathbf{h}_i^O$, $\mathbf{h}_k^M$, and $\mathbf{h}_{ik}^P$, respectively. The encoder consists of stacked BGNN layers with bidirectional cross-attention. We first describe the operation-node update, where operation nodes serve as queries and their eligible machine nodes serve as keys and values.

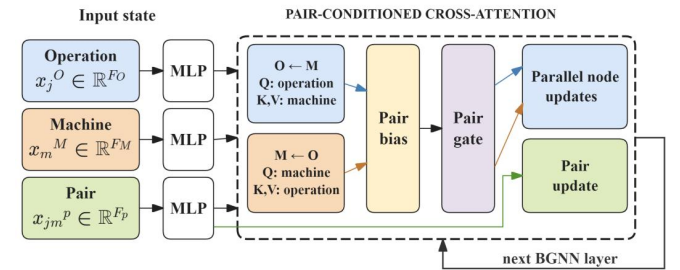


Figure 3. Architecture of one pair-conditioned BGNN layer.

For attention head r , the score and normalized weight between operation i and machine k are computed as:

$$e_{ik}^{(r)} = \frac{\mathbf{q}_i^{(r)\top} \mathbf{k}_k^{(r)}}{\sqrt{d_r}} + \mathbf{w}_a^{(r)\top} \mathbf{h}_{ik}^P \quad (2)$$

$$\alpha_{ik}^{(r)} = \text{softmax}_{k: D_{t,ik}=0} (e_{ik}^{(r)})$$

Here, $\mathbf{q}_i^{(r)}$ and $\mathbf{k}_k^{(r)}$ are the query and key vectors of head r , $\mathbf{h}_{ik}^P$ is the pair embedding, and $D_{t,ik} = 0$ indicates an eligible pair information into the attention score.

In addition to modifying the attention scores, the pair embeddings are used to gate the aggregated machine message:

$$\begin{aligned} \mathbf{c}_i^p &= \sum_k \bar{\alpha}_{ik} \mathbf{h}_{ik}^p \\ \mathbf{g}_i &= 1 + \tanh(\mathbf{W}_g \mathbf{c}_i^p) \\ \mathbf{m}_i^o &= \mathbf{g}_i \odot \mathbf{W}_o \left(\sum_{r=1}^H \alpha_{ik}^{(r)} \mathbf{v}_k^{(r)} \right) \end{aligned} \quad (3)$$

Here, $\bar{\alpha}_{ik}$ is the mean attention weight across heads, and $\mathbf{v}_k^{(r)}$ is the value vector of machine k in head r . Thus, pair information affects both attention weighting and message aggregation.

The operation embedding is updated using the gated message, and the pair embedding is then updated using the new operation and machine embeddings:

$$\begin{aligned} \mathbf{h}_i^o &\leftarrow \text{LN}(\mathbf{h}_i^o + \mathbf{m}_i^o) \\ \mathbf{h}_{ik}^p &\leftarrow \text{LN}(\mathbf{h}_{ik}^p + \tanh(\mathbf{W}_p[\mathbf{h}_i^o \parallel \mathbf{h}_k^M \parallel \mathbf{h}_{ik}^p])) \end{aligned} \quad (4)$$

Machine nodes are updated in the same manner, with machines serving as queries and operations as keys and values. Finally, local projections are used for action scoring, while masked mean pooling produces the graph-level representation used by the critic and cross-step memory.

3.3 Cross-Step Memory with GRU

Scheduling decisions depend on context accumulated across earlier steps. As shown in Fig. 4, a gated recurrent unit (GRU) [12] combines the current graph-level representation with the previous hidden state to update the history representation. The hidden state is initialized to zero for each instance and reset at the end of an episode.

For each operation-machine pair (i, k) , the actor input is:

$$\mathbf{z}_{t,ik} = [\mathbf{a}_i^o \parallel \mathbf{a}_k^M \parallel \mathbf{a}_{ik}^p \parallel \mathbf{g}_i^o \parallel \mathbf{g}_k^M \parallel \mathbf{h}_t^{hist}] \quad (5)$$

which combines local pair information, pooled graph features, and the history representation. An MLP maps $\mathbf{z}_{t,ik}$ to an action logit, while the critic estimates the state value from the graph-level and history representations.

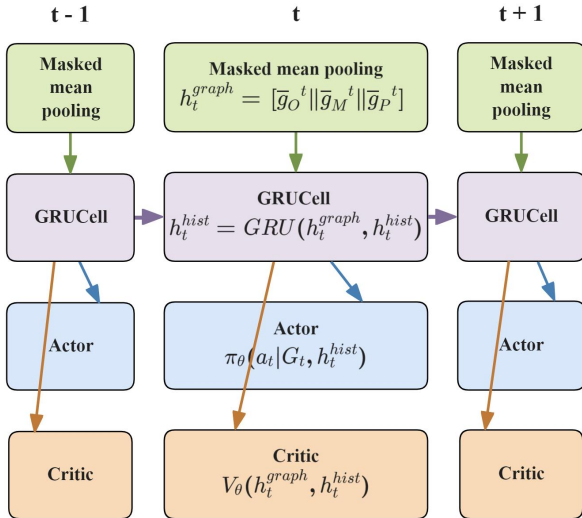


Figure 4. Cross-step information flow in the GRU-based actor-critic network.

3.4 Training

The encoder, GRU, actor, and critic are trained jointly using PPO. For a trajectory of length T , generalized advantage estimation (GAE) [13] is computed backward as

$$\begin{aligned} \delta_t &= r_t + \gamma(1 - d_t)V_{t+1} - V_t \\ A_t &= \delta_t + \gamma\lambda(1 - d_t)A_{t+1} \end{aligned} \quad (6)$$

where d_t indicates episode termination, and γ and λ are the discount and GAE parameters, respectively. The value target is defined as $\hat{R}_t = V_t + A_t$. To reduce the effect of different reward scales, the advantages are standardized over time for each parallel environment, while the value targets remain on the original reward scale.

The minimized PPO objective is

$$\begin{aligned} \mathcal{L}(\theta) &= -c_p \mathbb{E}_t [\min(\rho_t \tilde{A}_t, \text{clip}(\rho_t, 1 - \epsilon, 1 + \epsilon) \tilde{A}_t)] \\ &\quad + c_v \mathbb{E}_t [(V_\theta - \hat{R}_t)^2] - c_e \mathbb{E}_t [\mathcal{H}(\pi_\theta)] \end{aligned} \quad (7)$$

where ρ_t is the probability ratio between the current and old policies, $\tilde{A}_t$ is the standardized advantage, ϵ is the clipping parameter, and c_p, c_v, c_e are the loss coefficients.

During each PPO update, the stored trajectory is recomputed from a zero-initialized hidden state, allowing gradients to propagate through the GRU across decision steps. The termination mask resets the hidden state at episode boundaries.

4 Experiments and Analysis

4.1 Experimental Setup

The training and validation sets consist of synthetically generated FJSP instances with 10 jobs and 5 machines. The test set contains 130 public instances, including Mk01-Mk10 from Brandimarte [14] and the edata, rdata, and vdata subsets from Hurink [15].

The model is implemented in PyTorch 2.10.0 with Python 3.12.13 and evaluated on a workstation equipped with an Intel Core i7-8700 CPU and an NVIDIA GeForce RTX 2080 Ti GPU with 11 GB of memory. The encoder contains two BGNN layers, while the actor and critic use three-layer MLPs. We set $(c_p, c_v, c_e) = (1, 0.5, 0.01)$, $\gamma = 1$, $\lambda = 0.98$ and $\epsilon = 0.2$. Training uses Adam with a learning rate of 3×10^{-4} , 20 parallel environments, four PPO epochs per update, and 1,000 updates. Gradients are clipped to a maximum norm of 1.0. At inference, we use stochastic beam search with a beam width of 10.

We compare our method with four dispatching rules (SPT, FIFO, MOR, and MWKR) and three DRL baselines (LMLPF[7], HGNN[10], and DANIEL[11]). All baselines are reproduced under the same settings, except LMLPF, whose reported results are used because its source code is unavailable. Solution quality is evaluated using the relative gap between the obtained makespan C_{max} and the known upper bound C_{UB} :

$$\text{Gap} = \frac{C_{max} - C_{UB}}{C_{UB}} \times 100\% \quad (8)$$

Table 1 Performance on public benchmarks

Method	Brandimarte		Hurink rdata		Hurink edata		Hurink vdata	
	Gap	Time(s)	Gap	Time(s)	Gap	Time(s)	Gap	Time(s)
SPT	44.60%	0.28	29.43%	0.30	26.77%	0.30	18.15%	0.30
FIFO	31.42%	0.28	17.21%	0.30	20.78%	0.30	7.54%	0.30
MOR	27.82%	0.29	15.02%	0.30	19.21%	0.30	5.64%	0.30
MWKR	28.61%	0.28	13.81%	0.30	18.57%	0.30	4.18%	0.30
LMLPF	14.04%	0.41	12.14%	0.27	15.54%	0.27	5.35%	0.27
HGNN	28.52%	1.75	11.15%	1.86	15.53%	1.88	4.25%	1.88
DANIEL	13.58%	1.44	11.42%	1.54	16.33%	1.53	3.28%	1.55
Ours	20.67%	1.08	7.83%	1.15	14.47%	1.15	2.59%	1.15

4.2 Performance on Public Benchmarks

We compare our method with four handcrafted PDRs and three DRL baselines, as summarized in Table 1. HGNN and DANIEL were reproduced using 10×5 models with greedy decoding, whereas the LMLPF results were taken from the original publication. HGNN uses heterogeneous graph encoding, DANIEL employs dual attention over operation and machine representations, and LMLPF directly encodes candidate actions using a lightweight MLP.

Our method achieved the lowest mean Gap on all three Hurink subsets. This performance supports the use of a compact bipartite representation that focuses message passing on currently available operation-machine pairs, while pair-conditioned attention incorporates assignment-specific information. Although DANIEL and LMLPF performed better on Brandimarte, our method showed a clear advantage across the Hurink distributions.

Among the methods reproduced under the same settings, the proposed method required less inference time than HGNN and DANIEL but remained slower than the dispatching rules. The reported runtime of LMLPF is not directly comparable because its results were taken from the original publication.

Figure 5 further shows lower and more concentrated Gap distributions on rdata and vdata, indicating reliable performance across individual instances. The differences are less pronounced on edata, whereas the Brandimarte distributions reflect the trend observed in Table 1. Our method also recorded lower inference times than HGNN and DANIEL.

4.3 Ablation study

To assess the contribution of each component, we compare the full model with three variants: Mean-Agg replaces attention-weighted aggregation with uniform averaging, w/o Pair Bias removes the pair-dependent attention term, and w/o GRU removes cross-step memory. As shown in Table 2, the full model achieves the lowest mean Gap on all four benchmark groups. Mean-Agg consistently degrades solution quality, supporting the adaptive weighting of eligible neighbors. Removing the pair bias has its clearest effect on vdata, while removing the GRU causes the largest overall degradation, particularly on vdata. These results confirm that attention weighting, pair information, and recurrent context all contribute to the final performance.

Table 2 Ablation results on the public benchmarks.

Dataset	Mean-agg	No Pair Bias	No GRU	Full
Brandi marte	25.44 ± 19.85	22.26 ± 17.85	26.67 ± 23.46	20.67 ± 15.78
Hurink rdata	9.26 ± 6.31	8.36 ± 5.15	10.87 ± 9.06	7.83 ± 5.47
Hurink edata	18.34 ± 8.33	14.59 ± 5.61	15.02 ± 7.26	14.47 ± 7.40
Hurink vdata	2.69 ± 1.79	3.91 ± 2.41	10.12 ± 13.75	2.57 ± 1.59

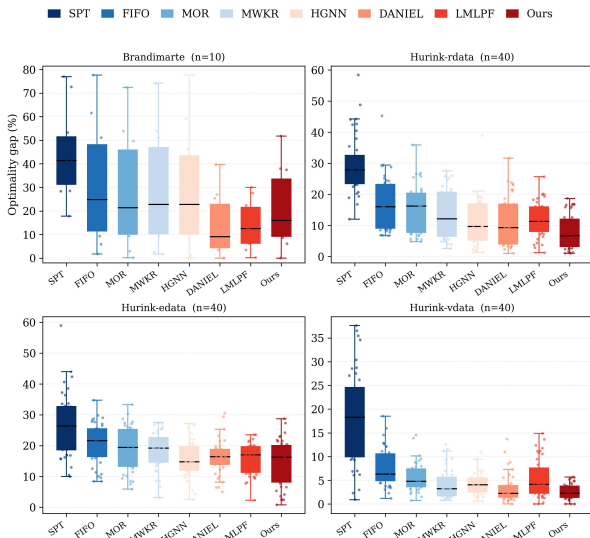


Figure 5. Relative Gap distributions of handcrafted PDRs and DRL policies on the public benchmarks.

Figure 6 shows that all variants improve rapidly during the early stage of training, with clearer differences emerging later. The full model maintains a low and relatively stable validation makespan, whereas w/o GRU exhibits larger late-stage fluctuations. Although w/o Pair Bias remains close to the full model on the validation set, its weaker performance on the public benchmarks indicates that the pair-dependent bias contributes to generalization across instance distributions.

Overall, the benchmark and validation results support the complete architecture.

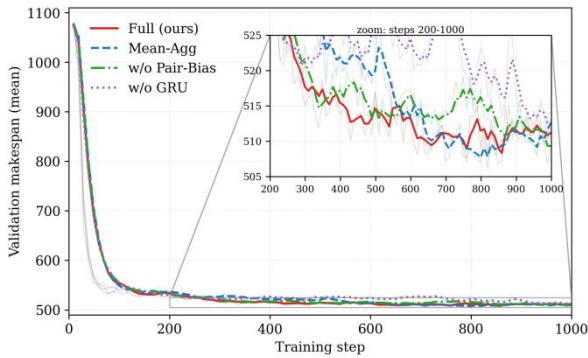


Figure 6. Validation makespan of the full model and three ablation variants over 1,000 training updates.

5 Conclusion

This paper presents an end-to-end DRL framework for FJSP based on a compact operation-machine bipartite graph. The representation focuses on currently available assignments, while pair-conditioned bidirectional attention models operation-machine interactions and GRU-based memory maintains scheduling context across decisions. Experiments on public benchmarks show that the proposed method achieves the best results on all three Hurink subsets and remains competitive on Brandimarte, with lower inference time than the reproduced graph-based DRL baselines. The ablation results further confirm the contributions of attention-weighted aggregation, pair information, and cross-step memory. These findings show that compact state representation and temporal context support effective scheduling decisions. Future work will extend the framework to dynamic and stochastic scheduling environments and investigate its applicability to larger-scale industrial problems.

6 References

- [1] J. Chen, J. Sun, and G. Wang, 'From unmanned systems to autonomous intelligent systems', *Engineering*, vol. 12, pp. 16 – 19, May 2022, doi: 10.1016/j.eng.2021.10.007.
- [2] I. A. Chaudhry and A. A. Khan, 'A research survey: review of flexible job shop scheduling techniques', *International Transactions in Operational Research*, vol. 23, no. 3, pp. 551 – 591, 2016, doi: 10.1111/itor.12199.
- [3] S. Dautère-Pérès, J. Ding, L. Shen, and K. Tamssaouet, 'The flexible job shop scheduling problem: A review', *European Journal of Operational Research*, vol. 314, no. 2, pp. 409 – 432, Apr. 2024, doi: 10.1016/j.ejor.2023.05.017.
- [4] I. G. Smit et al., 'Graph neural networks for job shop scheduling problems: A survey', *Comput. Oper. Res.*, vol. 176, p. 106914, Apr. 2025, doi: 10.1016/j.cor.2024.106914.
- [5] C. Zhang, W. Song, Z. Cao, J. Zhang, P. S. Tan, and C. Xu, 'Learning to dispatch for job shop scheduling via deep reinforcement learning', in *Advances in Neural Information Processing Systems*, vol. 33, pp. 1621-1632, 2020.
- [6] S. Luo, 'Dynamic scheduling for flexible job shop with new job insertions by deep reinforcement learning', *Applied Soft Computing*, vol. 91, Article 106208, 2020, doi: 10.1016/j.asoc.2020.106208.
- [7] Yuan, E., Wang, L., Cheng, S., et al.: 'Solving flexible job shop scheduling problems via deep reinforcement learning', *Expert Syst. Appl.*, 2024, 245, Article 123019. doi: 10.1016/j.eswa.2023.123019
- [8] J. Park, J. Chun, S. H. Kim, et al. 'Learning to schedule job-shop problems: representation and policy learning using graph neural network and reinforcement learning', *International Journal of Production Research*, vol. 59, no. 11, pp. 3360-3377, 2021, doi: 10.1080/00207543.2020.1870013.
- [9] K. Lei, P. Guo, W. Zhao, et al. 'A multi-action deep reinforcement learning framework for flexible job-shop scheduling problem', *Expert Systems with Applications*, vol. 205, Article 117796, 2022, doi: 10.1016/j.eswa.2022.117796.
- [10] W. Song, X. Chen, Q. Li, and Z. Cao, 'Flexible job-shop scheduling via graph neural network and deep reinforcement learning', *IEEE Trans. Ind. Inf.*, vol. 19, no. 2, pp. 1600 – 1610, Feb. 2023, doi: 10.1109/TII.2022.3189725.
- [11] R. Wang, G. Wang, J. Sun, et al. 'Flexible job shop scheduling via dual attention network-based reinforcement learning', *IEEE Trans. Neural Netw. Learning Syst.*, vol. 35, no. 3, pp. 3091-3102, Mar. 2024, doi: 10.1109/TNNLS.2023.3306421.
- [12] K. Cho et al., 'Learning phrase representations using RNN encoder-decoder for statistical machine translation', in *Proceedings of EMNLP*, pp. 1724–1734, 2014, doi: 10.3115/v1/D14-1179.
- [13] Schulman, J., et al.: 'High-dimensional continuous control using generalized advantage estimation'. *Proc. Int. Conf. Learning Representations (ICLR)*, San Juan, Puerto Rico, May 2016, arXiv:1506.02438.
- [14] P. Brandimarte, 'Routing and scheduling in a flexible job shop by tabu search', *Ann Oper Res*, vol. 41, no. 3, pp. 157–183, Sep. 1993, doi: 10.1007/BF02023073.
- [15] J. Hurink, B. Jurisch, and M. Thole, 'Tabu search for the job-shop scheduling problem with multi-purpose machines,' *OR Spektrum*, vol. 15, no. 4, pp. 205–215, Dec. 1994, doi: 10.1007/BF01719451.